

Why does Dijkstra's Algorithm fail on negative weights?

Data Structure

C++

Programming

Introduction

Dijkstra's algorithm could be a broadly utilized graph traversal algorithm that finds the shortest way between two vertices in a graph. It is effective and ensures ideal comes about when connected to graphs with non-negative weights. In any case, when negative weights are presented, Dijkstra's algorithm comes up short to produce rectify comes about. In this article, we'll investigate the reasons behind this disappointment and discuss three distinctive approaches to handling negative weights in graphs using C language. We'll give a step-by-step clarification of each approach, in conjunction with the comparing code and outputs.

Understanding Dijkstra's Algorithm

Dijkstra's algorithm works by iteratively expanding the shortest ways from the source vertex to all other vertices within the graph until the goal vertex is come to. It keeps up a needed line of vertices, known as the "unvisited set," and keeps track of the most limited known separate from the source to each vertex.

At first, the removal to the source vertex is set to zero, and all other separations are set to infinity. As the calculation advances, it chooses the vertex with the least distance from the unvisited set, investigates its neighboring vertices, and overhauls their separations in the event that a shorter way is found. This handle proceeds until the goal vertex is come to or there are no more vertices to visit.

Explore our **latest online courses** and learn new skills at your own pace. Enroll and become a certified expert to boost your career.

The Problem with Negative Weights

The most issue with negative weights in Dijkstra's algorithm lies within the assumption that the shortest way to any given vertex is continuously found by going by vertices in expanding order of separate from the source. However, when negative weights are presented, this suspicion breaks down, driving inaccurate results.

To get why this happens, let's consider a case where Dijkstra's calculation falls flat. Assume we have a graph with three vertices –

A, B, and C are connected as takes after $A \rightarrow (-1) \rightarrow B \rightarrow (-2) \rightarrow C$

Accepting the source vertex may be, a Dijkstra's algorithm will at first set the remove to B as -1 and the separate to C as boundlessness. In any case, when we consider the edge from B to C with weight -2, the algorithm will inaccurately overhaul the separate to C as -3, expecting the way $A \rightarrow B \rightarrow C$ is shorter than the current way $A \rightarrow C$.

The algorithm comes up short to recognize that the negative weight of -2 assist diminishes the removal from B to C, making the way $A \rightarrow C$ shorter than $A \rightarrow B \rightarrow C$. This inaccurate upgrade leads to an erroneous shortest-way calculation.

Code Implementation in C Language

Let's presently look at a code execution of Dijkstra's algorithm within the C language, highlighting its confinements when confronted with negative weights.

Example

[Open Compiler](#)

```
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>

#define G 4

int short_dst(int dt[], bool visited[]) {
    int min = INT_MAX, x;

    for (int t = 0; t < G; t++)
        if (visited[t] == false && dt[t] < min)
            min = dt[t], x = t;

    return x;
}

void dkst(int graph[G][ G], int src) {
    int dist[G];
    bool visited[G];

    for (int i = 0; i < G; i++) {
        dist[i] = INT_MAX;
        visited[i] = false;
    }

    dist[src] = 0;
```

```
for (int count = 0; count < G - 1; count++) {
    int u = short_dst(dist, visited);
    visited[u] = true;

    for (int t = 0; t < G; t++) {
        if (!visited[t] && graph[u][t] && dist[u] != INT_MAX && dist[u] + graph[u][t]
< dist[t])
            dist[t] = dist[u] + graph[u][t];
    }
}

printf("V\t Distance from Source\n");
for (int i = 0; i < G; i++)
    printf("%d\t%d\n", i, dist[i]);
}

int main() {
    int graph[G][ G] = {{0, 1, INT_MAX, INT_MAX},
                        {INT_MAX, 0, INT_MAX, INT_MAX},
                        {INT_MAX, INT_MAX, 0, -2},
                        {INT_MAX, INT_MAX, INT_MAX, 0}};

    dkst(graph, 0);

    return 0;
}
```

Output

V	Distance from Source
0	0
1	1
2	-2147483648
3	-2147483648

Conclusion

Dijkstra's algorithm is an effective and dependable algorithm for finding the briefest way in graphs with non-negative weights. In any case, it falls flat to create exactly what comes about when the chart contains negative weights. The inaccurate presumptions made by the algorithm when managing with negative weights lead to wrong shortest-way calculations whereas Dijkstra's

algorithm has its confinements when it comes to negative weights, it remains an important instrument in scenarios where non-negative weights are displayed. By understanding the reasons behind its disappointment and investigating elective algorithms like Bellman-Ford, ready to make educated choices when selecting the foremost fitting calculation for fathoming graph-related issues, guaranteeing proficiency and exact comes about in different spaces.